

C-EASY-4：小应用之五子棋游戏

Step2.引入坐标，绘制棋盘

一、字符坐标到棋盘坐标

每一行，每一列下标都是从0开始，于是容易得到：

+：[2i] [4j] (i,j=0,1,2,3...)

-：[2i] [4j+1], [2i] [4j+2], [2i] [4j+3] (i,j=0,1,2,3...)

|：[2i+1] [4j] (i,j=0,1,2,3...)

.：[2i+1] [4j+2] (i,j=0,1,2,3...)

每个位置的行列数都有多种表示方法，最好能使 i 和 j 取值范围保持统一

二、实现下棋功能

1.初始化棋盘

初始化棋盘，本质上是对二维数组中的每个元赋值，由前面学习过的数组可推知需要用到遍历，而且是嵌套遍历，注意到当 i 相同时，- 和 . 的下标总是大于+和|，当 j 相同时，| 和 . 的下标总是大于+和-，然而棋盘的边界是由后者组成的，这对遍历提出了更高的要求，使我不得不去添加更多上界的条件（或者限制 i 和 j 下界，但这样就需要更改| 和 . 的下标通式，于是我知难而退），| 需要限制行数，- 需要限制列数，. 既需要限制行数也需要限制列数

利用cell指针列表，我完成了 . 位置的简化，这对使用API定位带来了极大的便利

2.绘制棋盘

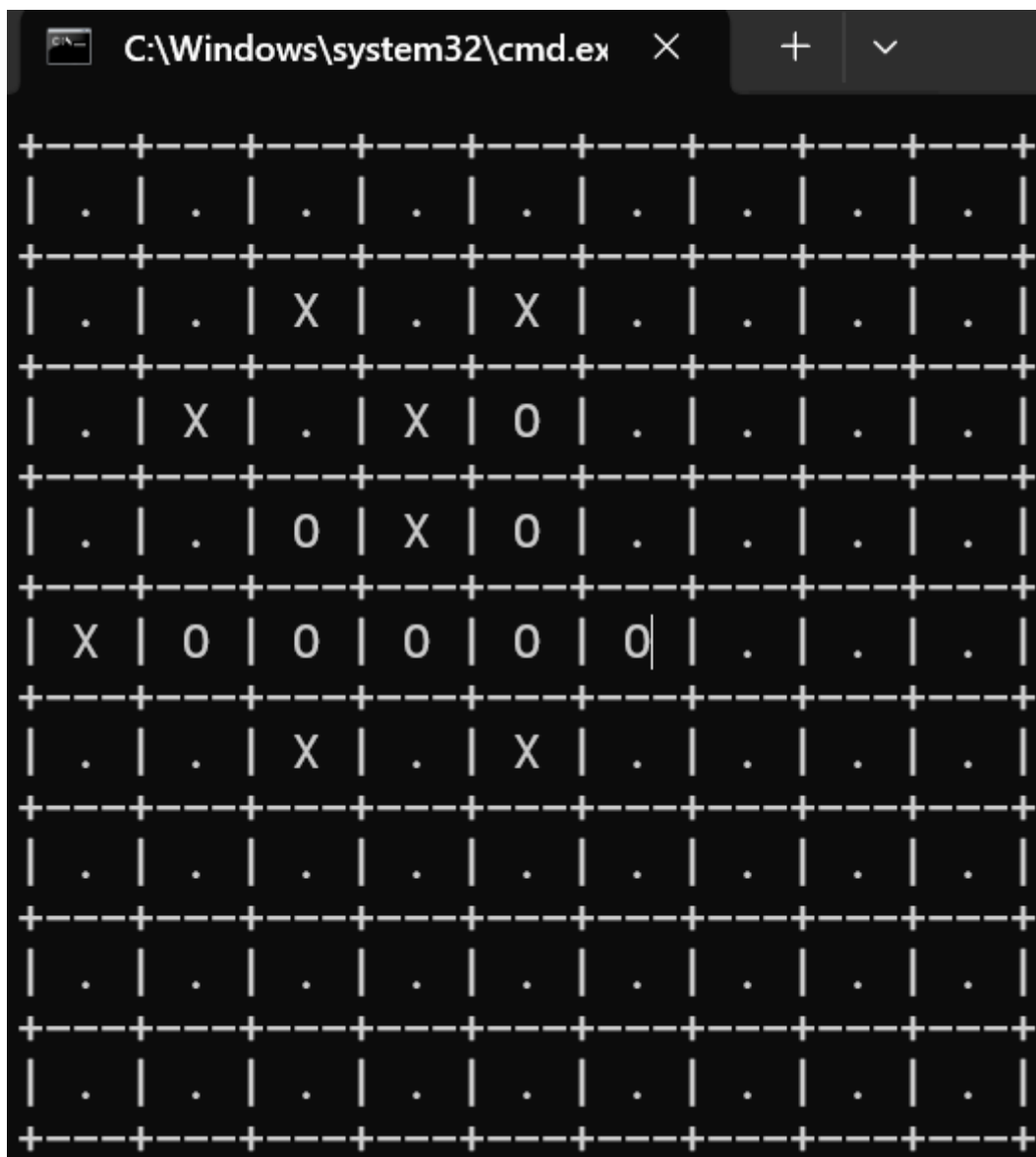
在绘制棋盘时，根据终端横向输出的特性，i 的for循环应当嵌套在 j 的for当中，同时，要记得换行。此外，不仅需要考虑到赋字符的位置，还要想到在没有赋字符的位置输出空格

在综合考虑之下，便有了：



3.&4.

说来惭愧，Windows的原生API实在是太复杂，只能请求一下AI的帮助了，当然AI肯定没有智能到完美完成任务（可能是因为我没付费吧），我在基本了解API的前提下，指挥AI迭代了cells与物理坐标的对应，利用浮点数特性扩大光标点击的识别范围，以及覆盖绘制棋盘等等功能，旨在更加完善五子棋的各种功能。当然在这之后，我也基本理解了API代码，相信这将会帮助我更好地完成接下来的部分



Step3.

一、

1.判定游戏是否结束

需要在上述的第四步“重新绘制棋盘”，也就是落子结果显示时，对cells数组进行检测，进而判定游戏状态

2.检测方法

设计棋盘坐标和字符坐标两种坐标的原因：字符坐标对应控制台的实际输出位置；棋盘坐标专门用于胜负的判定。字符坐标是打印棋盘和计算棋盘坐标的基础，棋盘坐标可以在判定游戏状态时降低耦合性，很好地避免复杂的表达式，

cells数组的检测

在我看来，最简单粗暴的方法就是遍历，无外乎四种情况：— | \ /

不想止步于此，我联想到上一题中的递归，它给我留下了十分深刻的印象，能不能通过递归来判定呢？

通过传入指针，可以追踪和比对外层与内层坐标指针的状态，从而判断是否存在连续的 X 或 O，配合一个计数器，当方向匹配时计数自增，一旦方向中断则清零

然而理想是丰满的，现实是骨感的，递归路上会遇到的问题数不胜数：递归方向的多样使得头指针位置的选择成为了巨大的难题，指针数量的控制也让人头疼：参数太少会导致状态高度耦合，一处清零可能会误伤其他方向的判定结果；参数太多又很可能会带来更多的问题。此外，我还担心有栈溢出的风险...

嗯，那就遍历吧

然而，遍历的辛酸只有敲过的人才明白，太冗长啦，这样的代码要重复4遍！

```
1  if(*cells[i][j] != '.' &&
2      *cells[i][j] == *cells[i+1][j] &&
3      *cells[i][j] == *cells[i+2][j] &&
4      *cells[i][j] == *cells[i+3][j] &&
5      *cells[i][j] == *cells[i+4][j]){
```

我还是问问AI吧...

AI的方法非常巧妙：以最后落子的位置为准，向八个方向分别检查是否有连珠的情况，如果相应两个方向之和不小于5或者走满81步，游戏就结束了

通过矩阵规定出最基本的四个方向

```
1  int directions[4][2] = {
2      {0, 1}, // 向右
3      {1, 0}, // 向下
4      {1, 1}, // 向右下
5      {1, -1} // 向左下
6  };
```

然后通过变量 i 来使cells的行数和列数一一对应

```
1  int dr = directions[d][0]; // 获取行增量
2      int dc = directions[d][1]; // 获取列增量
3  for (int i = 1; i < WIN_LENGTH; i++) {
4      int r = row + i * dr;
5      int c = col + i * dc;
6      if (r < 0 || r >= BOARD_SIZE ||
7          c < 0 || c >= BOARD_SIZE ||
8          *(game->cells[r][c]) != *(game->cells[row][col])) {
9          break;
10     }
11     count++;
12 }
```

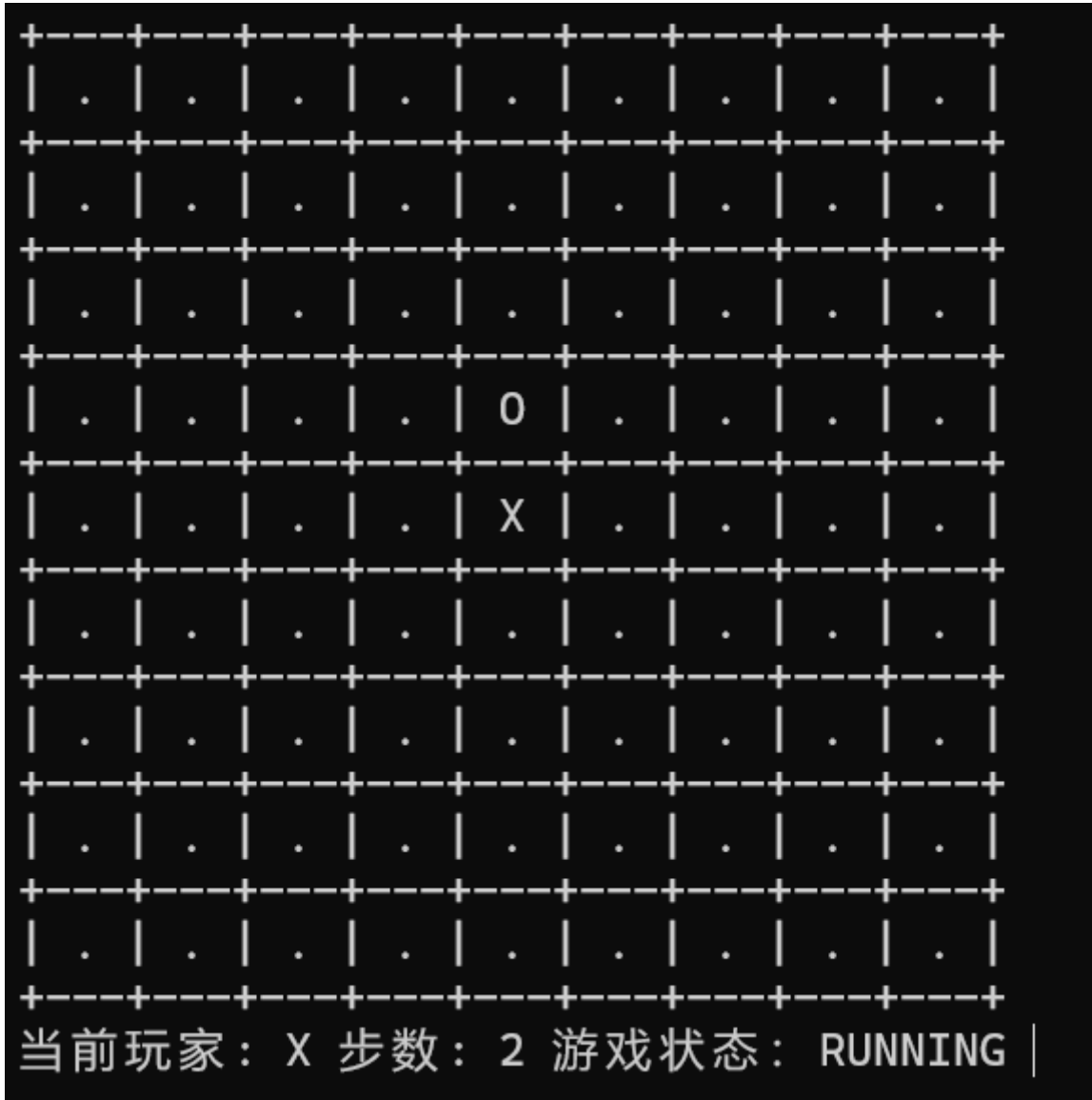
通过改变 i 的符号使检测反向

```
1  for (int i = 1; i < WIN_LENGTH; i++) {
2      int r = row - i * dr;
3      int c = col - i * dc;
4      if (r < 0 || r >= BOARD_SIZE ||
5          c < 0 || c >= BOARD_SIZE ||
6          *(game->cells[r][c]) != *(game->cells[row][col])) {
7          break;
8      }
9      count++;
10 }
```

3.提示语句

提示语句添加在棋盘下面的原因：如果添加在棋盘上面，就会使得棋盘打印的起点会随着提示语句的行数的改变而改变，添加在棋盘下面，想写多少就写多少，当然也更加符合阅读习惯

每一次落子之后光标的位置都会发生改变，这时就需要在刷新棋盘的同时刷新光标位置，并输出提示语句



同时还可以加上一些小互动：

```
+---+---+---+---+---+---+---+---+
|. |. |. |. |. |. |. |. |. |. |
+---+---+---+---+---+---+---+---+
|. |. |. |. |. |. |X |. |. |. |
+---+---+---+---+---+---+---+---+
|. |. |. |X |O |O |O |O |X |
+---+---+---+---+---+---+---+---+
|. |X |X |O |O |X |. |O |X |
+---+---+---+---+---+---+---+---+
|X |O |O |X |X |X |X |O |. |
+---+---+---+---+---+---+---+---+
|. |O |O |X |. |X |O |X |. |
+---+---+---+---+---+---+---+---+
|X |. |O |X |. |O |. |. |. |
+---+---+---+---+---+---+---+---+
|O |O |O |O |O |X |. |. |. |
+---+---+---+---+---+---+---+---+
|. |. |X |X |. |. |. |. |. |
+---+---+---+---+---+---+---+---+
===== O 获胜! =====
```

想要悔棋吗，少年？ y/n|

```

+---+---+---+---+---+---+---+---+---+
| . | . | . | . | . | . | . | . | . |
+---+---+---+---+---+---+---+---+---+
| . | . | . | . | . | X | . | . | . |
+---+---+---+---+---+---+---+---+---+
| . | . | . | X | O | O | O | O | X |
+---+---+---+---+---+---+---+---+---+
| . | X | X | O | O | X | . | O | X |
+---+---+---+---+---+---+---+---+---+
| X | O | O | X | X | X | X | O | . |
+---+---+---+---+---+---+---+---+---+
| . | O | O | X | . | X | O | X | . |
+---+---+---+---+---+---+---+---+---+
| X | . | O | X | . | O | . | . | . |
+---+---+---+---+---+---+---+---+---+
| O | O | O | O | O | X | . | . | . |
+---+---+---+---+---+---+---+---+---+
| . | . | X | X | . | . | . | . | . |
+---+---+---+---+---+---+---+---+---+
===== O 获胜! =====

```

想要悔棋吗，少年？ y/n
想P吃！哈哈
按任意键退出...|

后记

在这之后，我在B站上找到了贪吃蛇的项目视频，结合我自己的实操经验，对实际项目（下面以贪吃蛇举例）的完成有了更加清晰且整体的认识

- 先明确大致方向和主题思路，着重思考难点的突破方向（用什么方式定义贪吃蛇，如何移动贪吃蛇，如何实现改变蛇头方向的同时移动贪吃蛇）
- 然后敲定具体的功能板块以及各自的顺序（初始化游戏区域、贪吃蛇，定时刷新贪吃蛇和苹果，检测蛇头，移动贪吃蛇，胜负判定）
- 接着定义结构体、全局变量、函数声明
- 之后补充函数内容（初始化函数，检测蛇头函数，判定胜负函数...），攻克重难点（计时函数，蛇的移动函数...）
- 分模块测试与调试
- 最后还要根据效果对各个功能进行调整，使之达到预计的效果

就像是建一座房子，自下而上，由外而内。我想这种分析、解决问题的条理性与逻辑性正是我在做五子棋游戏的时候最缺乏的东西了（虽然有题目的引导，但是在实操的时候难免会手忙脚乱，顾此失彼，浪费了很多时间）

此外，我同样也加深了关于函数，指针，循环等等的了解，虽然现在还称不上是如臂使指，游刃有余，但也勉强能说是熟练了

总的来说，这对我算是一个不小的挑战，无论是编写函数、还是实现模块间的契合，以及调用API...需要考虑的事情太多，每一个困难都让我头疼不已，应接不暇，可以说，没有AI，我几乎不可能在短时间内完成这个任务，赞美AI，赞美五子棋，也赞美出题人，让我在敲代码的过程中学到了很多。当然，在获得全新体验的同时，我也意识到了自己的许多不足，嗯，我的路还很长，也许下个星期，也许下个月，当我再次打开这个文件夹的时候，我又有了敲代码添加更多功能的冲动...